Data Detective

Build a reliable cleaning and analysis program for the supplied fictional game records. Produce a console dashboard, an audit trail and numerical evidence for two questions.

Complete the prep HTML first. Allow 5–7 lessons of 90 minutes, or longer if needed. Finish all six stages; save a working version after each.

Stage 1 Read the file and understand each record

Do this

1. Inspect game_matches.csv. Each row is one player-match observation, not a complete independent match.

2. Use csv.DictReader. Expect record_id, match_id, player, character, kills, deaths, damage, won.

3. Report missing files, empty files and incorrect headers clearly. Preserve the source CSV.

4. Store rows in an explainable structure, such as a list of dictionaries. Keep original fields and each record's starting source line for the audit.

5. Develop with hand_check.csv first. The header is not a data record; the first ordinary record begins on line 2.

Check before moving on

You can display a few named fields correctly, report file/header problems without an unexplained crash, and explain the difference between record_id and match_id.

Watch out

A CSV is not always safe to read using split(","). A row is a player observation, not necessarily a whole independent match.

Stage 2 Normalise and validate individual rows

Do this

1. Trim whitespace. Require non-empty record_id, match_id and player. State a consistent case policy for IDs and names.

2. Normalise character case to Ranger, Mage, Tank or Assassin. Reject unknown characters.

3. Convert kills/deaths to whole numbers 0–50 and damage to 0–20000. Reject missing, decimal, non-numeric and out-of-range values.

4. Convert trimmed, case-insensitive true/false to booleans. Reject other won values.

5. Reject incomplete, extra-field and malformed rows with original fields, source line and reason. Continue where recovery is possible.

6. Keep high values within range: unusual does not mean invalid.

Check before moving on

The formatted ranger and TRUE example in hand_check.csv becomes a valid normalised row. Negative kills, Wizard, banana damage, blank player and yes for won are rejected with reasons.

Watch out

Converting a non-empty string directly to bool makes "false" true. Unusual but valid data must not be deleted as a validation error.

Stage 3 Resolve duplicates and reconcile the audit

Do this

1. Group valid normalised rows by record_id and compare all fields.

2. Identical copies: accept one, reject every extra copy. Formatting-only differences do not make a conflict.

3. Conflicting valid copies: quarantine all copies, including an earlier accepted one. Do not average or silently choose one.

4. Reject invalid rows for their own reason. An invalid copy alone does not invalidate a separate valid row with that ID.

5. Count every loaded record once: loaded = accepted + rejected. Verify the 14-row fixture before the full file.

Check before moving on

hand_check.csv gives 14 loaded = 3 accepted + 11 rejected. Keep one R1, keep R2 and R4, reject the extra R1 and all three conflicting R3 rows. Every rejected record has a source line and reason.

Watch out

Rejecting only a later conflicting copy still leaves an unreliable earlier row in the statistics. Identical duplicate observations must not count twice.

Stage 4 Calculate character and player statistics

Do this

1. Group accepted rows by character. Report count, mean kills, mean damage and win percentage.

2. Use each group's count as its denominator. Win percentage = winning records ÷ group records × 100.

3. Group by player. Report count, win percentage and adjusted K/D = total kills ÷ max(1, total deaths).

4. Handle empty groups without division by zero. Show no data or clearly label placeholders.

5. Check Alice's two accepted fixture rows by hand. Keep calculation precision; round for display. Do not average individual K/D ratios.

Check before moving on

Alice's fixture results are count 2, mean kills 15, mean damage 200, win percentage 50% and adjusted K/D 30. Her two rows also give those Ranger group statistics.

Watch out

The ratio of totals is not the mean of individual ratios. The denominator must be the current group's accepted record count.

Stage 5 Build a dashboard and explainable shortlist

Do this

1. Build a repeating menu: character summaries, player lookup, anomaly shortlist and exit.

2. Validate choices and handle unknown names. Use accepted data and your documented name-case policy.

3. Define an explainable rule, such as mean damage strictly above 12000 AND at least five accepted records.

4. Show every flagged player’s value, sample size and reason. A flag requests investigation; it does not prove cheating.

5. Test four versus five records and an exact-threshold mean. If no player qualifies, show a clear empty-shortlist message.

Check before moving on

Every menu option works using accepted records. Unknown names and invalid choices are handled. Your threshold and minimum-sample tests demonstrate both flagged and unflagged cases.

Watch out

A high performance value is not proof of cheating. An empty shortlist can be the correct result.

Stage 6 Export evidence and investigate two questions

Do this

1. Export cleaned_matches.csv with the eight source fields; rejected_rows.csv with original fields, source_line and reason; summary_results.csv with labelled statistics.

2. Reopen exports and check headers, counts and values. Preserve the source and include a balanced audit.

3. Investigate two questions with numbers and sample sizes: e.g. highest observed character win rate, and mean kills in wins versus losses.

4. Explain limitations: player observations are not independent match wins; association does not prove causation.

5. Submit code, README, exports, audit, findings, expected/actual tests and a 200–300 word reflection: actual bug, fix, two tests, limitation and help used.

Check before moving on

The exports reopen correctly and your audit balances. Both questions have numerical answers and sensible limitations. Optional stretch: compare your aggregation with Pandas or investigate correlation after the required work.

Watch out

Association does not establish causation. Multiple player records from one match are not independent match outcomes.